

COMPRESSION SOURCE CODE

```
/*++
```

Copyright (c) 2001-2002 Intel Corporation

Module Name:

Compress.c

Abstract:

Compression routine. The compression algorithm **is** a mixture of LZ77 **and** Huffman Coding. LZ77 transforms the source data into a sequence of Original Characters **and** Pointers to repeated strings. This sequence **is** further divided into Blocks **and** Huffman codings are applied to each Block.

Revision History:

```
--*/
```

```
#include <string.h>
#include <stdlib.h>
#include "eficommon.h"
```

```
//
// Macro Definitions
//
```

```
typedef INT16      NODE;
#define UINT8_MAX   0xff
#define UINT8_BIT    8
#define THRESHOLD    3
#define INIT_CRC     0
#define WNDBIT       13
#define WNDISZ       (1U << WNDBIT)
#define MAXMATCH     256
#define PERC_FLAG    0x8000U
#define CODE_BIT     16
#define NIL          0
#define MAX_HASH_VAL (3 * WNDISZ + (WNDISZ / 512 + 1) * UINT8_MAX)
#define HASH(p, c) ((p) + ((c) << (WNDBIT - 9)) + WNDISZ * 2)
#define CRCPOLY      0xA001
```

(continues on next page)

(continued from previous page)

```
#define UPDATE_CRC(c) mCrc = mCrcTable[(mCrc ^ (c)) & 0xFF] ^ (mCrc
>> UINT8_BIT)

//
// C: the Char&Len Set; P: the Position Set; T: the exTra Set
//

#define NC      (UINT8_MAX + MAXMATCH + 2 - THRESHOLD)
#define CBIT    9
#define NP      (WNDBIT + 1)
#define PBIT    4
#define NT      (CODE_BIT + 3)
#define TBIT    5
#if NT > NP
    #define NPT NT
#else
    #define NPT NP
#endif

//
// Function Prototypes
//

STATIC
VOID
PutDword(
    IN UINT32 Data
);

STATIC
EFI_STATUS
AllocateMemory (
);

STATIC
VOID
FreeMemory (
);

STATIC
VOID
InitSlide (
);

STATIC
NODE
Child (
    IN NODE q,
    IN UINT8 c
);

STATIC
```

(continues on next page)

(continued from previous page)

```

VOID
MakeChild (
    IN NODE q,
    IN UINT8 c,
    IN NODE r
);

STATIC
VOID
Split (
    IN NODE old
);

STATIC
VOID
InsertNode (
);

STATIC
VOID
DeleteNode (
);

STATIC
VOID
GetNextMatch (
);

STATIC
EFI_STATUS
Encode (
);

STATIC
VOID
CountTFreq (
);

STATIC
VOID
WritePTLen (
    IN INT32 n,
    IN INT32 nbit,
    IN INT32 Special
);

STATIC
VOID
WriteCLen (
);

STATIC

```

(continues on next page)

(continued from previous page)

```

VOID
EncodeC (
    IN INT32 c
);

STATIC
VOID
EncodeP (
    IN UINT32 p
);

STATIC
VOID
SendBlock (
);

STATIC
VOID
Output (
    IN UINT32 c,
    IN UINT32 p
);

STATIC
VOID
HufEncodeStart (
);

STATIC
VOID
HufEncodeEnd (
);

STATIC
VOID
MakeCrcTable (
);

STATIC
VOID
PutBits (
    IN INT32 n,
    IN UINT32 x
);

STATIC
INT32
FreadCrc (
    OUT UINT8 *p,
    IN INT32 n
);
    
```

(continues on next page)

(continued from previous page)

```

STATIC
VOID
InitPutBits (
);

STATIC
VOID
CountLen (
    IN INT32 i
);

STATIC
VOID
MakeLen (
    IN INT32 Root
);

STATIC
VOID
DownHeap (
    IN INT32 i
);

STATIC
VOID
MakeCode (
    IN INT32 n,
    IN UINT8 Len[],
    OUT UINT16 Code[]
);

STATIC
INT32
MakeTree (
    IN INT32 NParm,
    IN UINT16 FreqParm[],
    OUT UINT8 LenParm[],
    OUT UINT16 CodeParm[]
);

//
// Global Variables
//

STATIC UINT8 *mSrc, *mDst, *mSrcUpperLimit, *mDstUpperLimit;

STATIC UINT8 *mLevel, *mText, *mChildCount, *mBuf, mCLen[NC], mPTLen[NPT], *mLen;
STATIC INT16 mHeap[NC + 1];
STATIC INT32 mRemainder, mMatchLen, mBitCount, mHeapSize, mN;
STATIC UINT32 mBufSiz = 0, mOutputPos, mOutputMask, mSubBitBuf, mCrc;
STATIC UINT32 mCompSize, mOrigSize;

```

(continues on next page)

(continued from previous page)

```
STATIC UINT16 *mFreq, *mSortPtr, mLenCnt[17], mLeft[2 * NC - 1], mRight[2 * NC - 1],
    mCrcTable[UINT8_MAX + 1], mCFreq[2 * NC - 1], mCTable[4096], mCCode[NC],
    mPFreq[2 * NP - 1], mPTCode[NPT], mTFreq[2 * NT - 1];
```

```
STATIC NODE mPos, mMatchPos, mAvail, *mPosition, *mParent, *mPrev, *mNext = NULL;
```

```
//
// functions
//
```

```
EFI_STATUS
```

```
Compress (
    IN UINT8 *SrcBuffer,
    IN UINT32 SrcSize,
    IN UINT8 *DstBuffer,
    IN OUT UINT32 *DstSize
)
```

```
/*++
```

Routine Description:

The main compression routine.

Arguments:

SrcBuffer - The buffer storing the source data
 SrcSize - The size of the source data
 DstBuffer - The buffer to store the compressed data
 DstSize - On **input**, the size of DstBuffer; On output,
 the size of the actual compressed data.

Returns:

EFI_BUFFER_TOO_SMALL - The DstBuffer **is** too small. In this case,
 DstSize contains the size needed.
 EFI_SUCCESS - Compression **is** successful.

```
--*/
{
    EFI_STATUS Status = EFI_SUCCESS;
```

```
//
// Initializations
//
```

```
mBufSiz = 0;
mBuf = NULL;
mText = NULL;
mLevel = NULL;
mChildCount = NULL;
```

(continues on next page)

(continued from previous page)

```

mPosition = NULL;
mParent = NULL;
mPrev = NULL;
mNext = NULL;

mSrc = SrcBuffer;
mSrcUpperLimit = mSrc + SrcSize;
mDst = DstBuffer;
mDstUpperLimit = mDst + *DstSize;

PutDword(0L);
PutDword(0L);

MakeCrcTable ();

mOrigSize = mCompSize = 0;
mCrc = INIT_CRC;

//
// Compress it
//

Status = Encode();
if (EFI_ERROR (Status)) {
    return EFI_OUT_OF_RESOURCES;
}

//
// Null terminate the compressed data
//
if (mDst < mDstUpperLimit) {
    *mDst++ = 0;
}

//
// Fill in compressed size and original size
//
mDst = DstBuffer;
PutDword(mCompSize+1);
PutDword(mOrigSize);

//
// Return
//

if (mCompSize + 1 + 8 > *DstSize) { \
    *DstSize = mCompSize + 1 + 8;
    return EFI_BUFFER_TOO_SMALL;
} else {
    *DstSize = mCompSize + 1 + 8;
    return EFI_SUCCESS;
}

```

(continues on next page)

(continued from previous page)

```

}

STATIC
VOID
PutDword(
    IN UINT32 Data
)
/*++

Routine Description:

    Put a dword to output stream

Arguments:

    Data - the dword to put

Returns: (VOID)

--*/
{
    if (mDst < mDstUpperLimit) {
        *mDst++ = (UINT8)(((UINT8)(Data )) & 0xff);
    }

    if (mDst < mDstUpperLimit) {
        *mDst++ = (UINT8)(((UINT8)(Data >> 0x08)) & 0xff);
    }

    if (mDst < mDstUpperLimit) {
        *mDst++ = (UINT8)(((UINT8)(Data >> 0x10)) & 0xff);
    }

    if (mDst < mDstUpperLimit) {
        *mDst++ = (UINT8)(((UINT8)(Data >> 0x18)) & 0xff);
    }
}

STATIC
EFI_STATUS
AllocateMemory ()
/*++

```

Routine Description:

Allocate memory spaces **for** data structures used **in** compression process

Arguments: (VOID)

Returns:

(continues on next page)

(continued from previous page)

```

    EFI_SUCCESS - Memory is allocated successfully
    EFI_OUT_OF_RESOURCES - Allocation fails

--*/
{
    UINT32 i;

    mText = malloc (WNSIZ * 2 + MAXMATCH);
    for (i = 0; i < WNSIZ * 2 + MAXMATCH; i++) {
        mText[i] = 0;
    }
    mLevel = malloc ((WNSIZ + UINT8_MAX + 1) * sizeof(*mLevel));
    mChildCount = malloc ((WNSIZ + UINT8_MAX + 1) *
        sizeof(*mChildCount));
    mPosition = malloc ((WNSIZ + UINT8_MAX + 1) * sizeof(*mPosition));
    mParent = malloc (WNSIZ * 2 * sizeof(*mParent));
    mPrev = malloc (WNSIZ * 2 * sizeof(*mPrev));
    mNext = malloc ((MAX_HASH_VAL + 1) * sizeof(*mNext));

    mBufSiz = 16 * 1024U;
    while ((mBuf = malloc(mBufSiz)) == NULL) {
        mBufSiz = (mBufSiz / 10U) * 9U;
        if (mBufSiz < 4 * 1024U) {
            return EFI_OUT_OF_RESOURCES;
        }
    }
    mBuf[0] = 0;
    return EFI_SUCCESS;
}

VOID
FreeMemory ()
/*++

```

Routine Description:

Called when compression **is** completed to free memory previously allocated.

Arguments: (VOID)

Returns: (VOID)

```

--*/
{
    if (mText) {
        free (mText);
    }

    if (mLevel) {
        free (mLevel);
    }
}

```

(continues on next page)

(continued from previous page)

```
if (mChildCount) {
    free (mChildCount);
}
```

```
if (mPosition) {
    free (mPosition);
}
```

```
if (mParent) {
    free (mParent);
}
```

```
if (mPrev) {
    free (mPrev);
}
```

```
if (mNext) {
    free (mNext);
}
```

```
if (mBuf) {
    free (mBuf);
}
```

```
return;
}
```

```
STATIC
VOID
```

```
InitSlide ()
/*++
```

Routine Description:

Initialize String Info Log data structures

Arguments: (VOID)

Returns: (VOID)

```
--*/
{
    NODE i;

    for (i = WNDSIZ; i <= WNDSIZ + UINT8_MAX; i++) {
        mLevel[i] = 1;
        mPosition[i] = NIL; /* sentinel */
    }
    for (i = WNDSIZ; i < WNDSIZ * 2; i++) {
        mParent[i] = NIL;
    }
}
```

(continues on next page)

(continued from previous page)

```
mAvail = 1;
for (i = 1; i < WNDISIZ - 1; i++) {
    mNext[i] = (NODE)(i + 1);
}

mNext[WNDISIZ - 1] = NIL;
for (i = WNDISIZ * 2; i <= MAX_HASH_VAL; i++) {
    mNext[i] = NIL;
}
}
```

```
STATIC
NODE
Child (
    IN NODE q,
    IN UINT8 c
)
/*++
```

Routine Description:

Find child node given the parent node **and** the edge character

Arguments:

q - the parent node
c - the edge character

Returns:

The child node (NIL **if not** found)

```
--*/
{
    NODE r;

    r = mNext[HASH(q, c)];
    mParent[NIL] = q; /* sentinel */
    while (mParent[r] != q) {
        r = mNext[r];
    }

    return r;
}
```

```
STATIC
VOID
MakeChild (
    IN NODE q,
    IN UINT8 c,
    IN NODE r
)
```

(continues on next page)

(continued from previous page)

```
/*++
```

Routine Description:

Create a new child **for** a given parent node.

Arguments:

q - the parent node
c - the edge character
r - the child node

Returns: (VOID)

```
--*/
```

```
{
    NODE h, t;

    h = (NODE)HASH(q, c);
    t = mNext[h];
    mNext[h] = r;
    mNext[r] = t;
    mPrev[t] = r;
    mPrev[r] = h;
    mParent[r] = q;
    mChildCount[q]++;
}
```

```
STATIC
VOID
Split (
    NODE Old
)
/*++
```

Routine Description:

Split a node.

Arguments:

Old - the node to split

Returns: (VOID)

```
--*/
```

```
{
    NODE New, t;

    New = mAvail;
    mAvail = mNext[New];
    mChildCount[New] = 0;
```

(continues on next page)

(continued from previous page)

```

    t = mPrev[Old];
    mPrev[New] = t;
    mNext[t] = New;
    t = mNext[Old];
    mNext[New] = t;
    mPrev[t] = New;
    mParent[New] = mParent[Old];
    mLevel[New] = (UINT8)mMatchLen;
    mPosition[New] = mPos;
    MakeChild(New, mText[mMatchPos + mMatchLen], Old);
    MakeChild(New, mText[mPos + mMatchLen], mPos);
}

```

STATIC

VOID

InsertNode ()

/*++

Routine Description:

Insert string info **for** current position into the String Info Log

Arguments: (VOID)

Returns: (VOID)

--*/

```

{
    NODE q, r, j, t;
    UINT8 c, *t1, *t2;

    if (mMatchLen >= 4) {

        //
        // We have just got a long match, the target tree
        // can be located by MatchPos + 1. Traverse the tree
        // from bottom up to get to a proper starting point.
        // The usage of PERC_FLAG ensures proper node deletion
        // in DeleteNode() later.
        //

        mMatchLen--;
        r = (INT16)((mMatchPos + 1) \ WNDISIZ);
        while ((q = mParent[r]) == NIL) {
            r = mNext[r];
        }
        while (mLevel[q] >= mMatchLen) {
            r = q; q = mParent[q];
        }
        t = q;
        while (mPosition[t] < 0) {
            mPosition[t] = mPos;

```

(continues on next page)

(continued from previous page)

```

    t = mParent[t];
}
if (t < WNDSIZ) {
    mPosition[t] = (NODE)(mPos \ | PERC_FLAG);
}
} else {

    //
    // Locate the target tree
    //

    q = (INT16)(mText[mPos] + WNDSIZ);
    c = mText[mPos + 1];
    if ((r = Child(q, c)) == NIL) {
        MakeChild(q, c, mPos);
        mMatchLen = 1;
        return;
    }
    mMatchLen = 2;
}

//
// Traverse down the tree to find a match.
// Update Position value along the route.
// Node split or creation is involved.
//

for ( ; ; ) {
    if (r >= WNDSIZ) {
        j = MAXMATCH;
        mMatchPos = r;
    } else {
        j = mLevel[r];
        mMatchPos = (NODE)(mPosition[r] & ~PERC_FLAG);
    }
    if (mMatchPos >= mPos) {
        mMatchPos -= WNDSIZ;
    }
    t1 = &mText[mPos + mMatchLen];
    t2 = &mText[mMatchPos + mMatchLen];
    while (mMatchLen < j) {
        if (*t1 != *t2) {
            Split(r);
            return;
        }
        mMatchLen++;
        t1++;
        t2++;
    }
    if (mMatchLen >= MAXMATCH) {
        break;
    }
}

```

(continues on next page)

(continued from previous page)

```
mPosition[r] = mPos;
q = r;
if ((r = Child(q, *t1)) == NIL) {
    MakeChild(q, *t1, mPos);
    return;
}
mMatchLen++;
}
t = mPrev[r];
mPrev[mPos] = t;
mNext[t] = mPos;
t = mNext[r];
mNext[mPos] = t;
mPrev[t] = mPos;
mParent[mPos] = q;
mParent[r] = NIL;

//
// Special usage of 'next'
//
mNext[r] = mPos;
}
```

```
STATIC
VOID
DeleteNode ()
/*++
```

Routine Description:

Delete outdated string info. (The Usage of PERC_FLAG ensures a clean deletion)

Arguments: (VOID)

Returns: (VOID)

```
--*/
{
    NODE q, r, s, t, u;

    if (mParent[mPos] == NIL) {
        return;
    }

    r = mPrev[mPos];
    s = mNext[mPos];
    mNext[r] = s;
    mPrev[s] = r;
    r = mParent[mPos];
    mParent[mPos] = NIL;
```

(continues on next page)

(continued from previous page)

```

if (r >= WNDSIZ || --mChildCount[r] > 1) {
    return;
}
t = (NODE)(mPosition[r] & ~PERC_FLAG);
if (t >= mPos) {
    t -= WNDSIZ;
}
s = t;
q = mParent[r];
while ((u = mPosition[q]) & PERC_FLAG) {
    u &= ~PERC_FLAG;
    if (u >= mPos) {
        u -= WNDSIZ;
    }
    if (u > s) {
        s = u;
    }
    mPosition[q] = (INT16)(s \ WNDSIZ);
    q = mParent[q];
}
if (q < WNDSIZ) {
    if (u >= mPos) {
        u -= WNDSIZ;
    }
    if (u > s) {
        s = u;
    }
    mPosition[q] = (INT16)(s \ WNDSIZ \ PERC_FLAG);
}
s = Child(r, mText[t + mLevel[r]]);
t = mPrev[s];
u = mNext[s];
mNext[t] = u;
mPrev[u] = t;
t = mPrev[r];
mNext[t] = s;
mPrev[s] = t;
t = mNext[r];
mPrev[t] = s;
mNext[s] = t;
mParent[s] = mParent[r];
mParent[r] = NIL;
mNext[r] = mAvail;
mAvail = r;
}

STATIC
VOID
GetNextMatch ()
/*++

```

Routine Description:

(continues on next page)

(continued from previous page)

Advance the current position (read in new data if needed).
Delete outdated string info. Find a match string for current position.

Arguments: (VOID)

Returns: (VOID)

```
--*/
{
    INT32 n;

    mRemainder--;
    if (++mPos == WNDISIZ * 2) {
        memmove(&mText[0], &mText[WNDISIZ], WNDISIZ + MAXMATCH);
        n = FreadCrc(&mText[WNDISIZ + MAXMATCH], WNDISIZ);
        mRemainder += n;
        mPos = WNDISIZ;
    }
    DeleteNode();
    InsertNode();
}
```

```
STATIC
EFI_STATUS
Encode ()
/*++
```

Routine Description:

The main controlling routine for compression process.

Arguments: (VOID)

Returns:

EFI_SUCCESS - The compression is successful
EFI_OUT_OF_RESOURCES - Not enough memory for compression process

```
--*/
{
    EFI_STATUS Status;
    INT32 LastMatchLen;
    NODE LastMatchPos;

    Status = AllocateMemory();
    if (EFI_ERROR(Status)) {
        FreeMemory();
        return Status;
    }
}
```

InitSlide();

(continues on next page)

(continued from previous page)

```

HufEncodeStart();

mRemainder = FreadCrc(&mText[WNDSIZ], WNDSIZ + MAXMATCH);

mMatchLen = 0;
mPos = WNDSIZ;
InsertNode();
if (mMatchLen > mRemainder) {
    mMatchLen = mRemainder;
}
while (mRemainder > 0) {
    LastMatchLen = mMatchLen;
    LastMatchPos = mMatchPos;
    GetNextMatch();
    if (mMatchLen > mRemainder) {
        mMatchLen = mRemainder;
    }

    if (mMatchLen > LastMatchLen || LastMatchLen < THRESHOLD) {

        //
        // Not enough benefits are gained by outputting a pointer,
        // so just output the original character
        //

        Output(mText[mPos - 1], 0);
    } else {

        //
        // Outputting a pointer is beneficial enough, do it.
        //

        Output(LastMatchLen + (UINT8_MAX + 1 - THRESHOLD),
            (mPos - LastMatchPos - 2) & (WNDSIZ - 1));
        while (--LastMatchLen > 0) {
            GetNextMatch();
        }
        if (mMatchLen > mRemainder) {
            mMatchLen = mRemainder;
        }
    }
}

HufEncodeEnd();
FreeMemory();
return EFI_SUCCESS;
}

STATIC
VOID
CountTFreq ()
    
```

(continues on next page)

(continued from previous page)

```
/*++
```

Routine Description:

Count the frequencies for the Extra Set

Arguments: (VOID)

Returns: (VOID)

```
--*/
{
    INT32 i, k, n, Count;

    for (i = 0; i < NT; i++) {
        mTFreq[i] = 0;
    }
    n = NC;
    while (n > 0 && mCLen[n - 1] == 0) {
        n--;
    }
    i = 0;
    while (i < n) {
        k = mCLen[i++];
        if (k == 0) {
            Count = 1;
            while (i < n && mCLen[i] == 0) {
                i++;
                Count++;
            }
            if (Count <= 2) {
                mTFreq[0] = (UINT16)(mTFreq[0] + Count);
            } else if (Count <= 18) {
                mTFreq[1]++;
            } else if (Count == 19) {
                mTFreq[0]++;
                mTFreq[1]++;
            } else {
                mTFreq[2]++;
            }
        } else {
            mTFreq[k + 2]++;
        }
    }
}
```

STATIC

VOID

WritePTLen (

IN INT32 n,
IN INT32 nbit,
IN INT32 Special

(continues on next page)

(continued from previous page)

```
)
/*++
```

Routine Description:

Outputs the code length array for the Extra Set or the Position Set.

Arguments:

n - the number of symbols
nbit - the number of bits needed to represent 'n'
Special - the special symbol that needs to be take care of

Returns: (VOID)

```
--*/
{
    INT32 i, k;

    while (n > 0 && mPTLen[n - 1] == 0) {
        n--;
    }
    PutBits(nbit, n);
    i = 0;
    while (i < n) {
        k = mPTLen[i++];
        if (k <= 6) {
            PutBits(3, k);
        } else {
            PutBits(k - 3, (1U << (k - 3)) - 2);
        }
        if (i == Special) {
            while (i < 6 && mPTLen[i] == 0) {
                i++;
            }
            PutBits(2, (i - 3) & 3);
        }
    }
}
```

```
STATIC
VOID
WriteCLen ()
/*++
```

Routine Description:

Outputs the code length array for Char&Length Set

Arguments: (VOID)

Returns: (VOID)

(continues on next page)

(continued from previous page)

```
--*/
{
    INT32 i, k, n, Count;

    n = NC;
    while (n > 0 && mClen[n - 1] == 0) {
        n--;
    }
    PutBits(CBIT, n);
    i = 0;
    while (i < n) {
        k = mClen[i++];
        if (k == 0) {
            Count = 1;
            while (i < n && mClen[i] == 0) {
                i++;
                Count++;
            }
            if (Count <= 2) {
                for (k = 0; k < Count; k++) {
                    PutBits(mPTLen[0], mPTCode[0]);
                }
            } else if (Count <= 18) {
                PutBits(mPTLen[1], mPTCode[1]);
                PutBits(4, Count - 3);
            } else if (Count == 19) {
                PutBits(mPTLen[0], mPTCode[0]);
                PutBits(mPTLen[1], mPTCode[1]);
                PutBits(4, 15);
            } else {
                PutBits(mPTLen[2], mPTCode[2]);
                PutBits(CBIT, Count - 20);
            }
        } else {
            PutBits(mPTLen[k + 2], mPTCode[k + 2]);
        }
    }
}

STATIC
VOID
EncodeC (
    IN INT32 c
)
{
    PutBits(mClen[c], mCCode[c]);
}

STATIC
VOID
EncodeP (
```

(continues on next page)

(continued from previous page)

```

    IN UINT32 p
    )
{
    UINT32 c, q;

    c = 0;
    q = p;
    while (q) {
        q >>= 1;
        c++;
    }
    PutBits(mPTLen[c], mPTCode[c]);
    if (c > 1) {
        PutBits(c - 1, p & (0xFFFFU >> (17 - c)));
    }
}

```

```

STATIC
VOID
SendBlock ()
/*++

```

Routine Description:

Huffman code the block and output it.

Argument: (VOID)

Returns: (VOID)

```

--*/
{
    UINT32 i, k, Flags, Root, Pos, Size;
    Flags = 0;

    Root = MakeTree(NC, mCFreq, mCLen, mCCode);
    Size = mCFreq[Root];
    PutBits(16, Size);
    if (Root >= NC) {
        CountTFreq();
        Root = MakeTree(NT, mTFreq, mPTLen, mPTCode);
        if (Root >= NT) {
            WritePTLen(NT, TBIT, 3);
        } else {
            PutBits(TBIT, 0);
            PutBits(TBIT, Root);
        }
        WriteCLen();
    } else {
        PutBits(TBIT, 0);
        PutBits(TBIT, 0);
    }
}

```

(continues on next page)

(continued from previous page)

```

    PutBits(CBIT, 0);
    PutBits(CBIT, Root);
}
Root = MakeTree(NP, mPFreq, mPTLen, mPTCode);
if (Root >= NP) {
    WritePTLen(NP, PBIT, -1);
} else {
    PutBits(PBIT, 0);
    PutBits(PBIT, Root);
}
Pos = 0;
for (i = 0; i < Size; i++) {
    if (i % UINT8_BIT == 0) {
        Flags = mBuf[Pos++];
    } else {
        Flags <<= 1;
    }
    if (Flags & (1U << (UINT8_BIT - 1))) {
        EncodeC(mBuf[Pos++] + (1U << UINT8_BIT));
        k = mBuf[Pos++] << UINT8_BIT;
        k += mBuf[Pos++];
        EncodeP(k);
    } else {
        EncodeC(mBuf[Pos++]);
    }
}
for (i = 0; i < NC; i++) {
    mCFreq[i] = 0;
}
for (i = 0; i < NP; i++) {
    mPFreq[i] = 0;
}
}

STATIC
VOID
Output (
    IN UINT32 c,
    IN UINT32 p
)
/*++

```

Routine Description:

Outputs an Original Character or a Pointer

Arguments:

c - The original character or the 'String Length' element of a Pointer
 p - The 'Position' field of a Pointer

Returns: (VOID)

(continues on next page)

(continued from previous page)

```

--*/
{
    STATIC UINT32 CPos;

    if ((mOutputMask >>= 1) == 0) {
        mOutputMask = 1U << (UINT8_BIT - 1);
        if (mOutputPos >= mBufSiz - 3 * UINT8_BIT) {
            SendBlock();
            mOutputPos = 0;
        }
        CPos = mOutputPos++;
        mBuf[CPos] = 0;
    }
    mBuf[mOutputPos++] = (UINT8) c;
    mCFreq[c]++;
    if (c >= (1U << UINT8_BIT)) {
        mBuf[CPos] |= mOutputMask;
        mBuf[mOutputPos++] = (UINT8)(p >> UINT8_BIT);
        mBuf[mOutputPos++] = (UINT8) p;
        c = 0;
        while (p) {
            p >>= 1;
            c++;
        }
        mPFreq[c]++;
    }
}

STATIC
VOID
HufEncodeStart ()
{
    INT32 i;

    for (i = 0; i < NC; i++) {
        mCFreq[i] = 0;
    }
    for (i = 0; i < NP; i++) {
        mPFreq[i] = 0;
    }
    mOutputPos = mOutputMask = 0;
    InitPutBits();
    return;
}

STATIC
VOID
HufEncodeEnd ()
{
    SendBlock();
}

```

(continues on next page)

(continued from previous page)

```
//
// Flush remaining bits
//
PutBits(UINT8_BIT - 1, 0);

return;
}

STATIC
VOID
MakeCrcTable ()
{
    UINT32 i, j, r;

    for (i = 0; i <= UINT8_MAX; i++) {
        r = i;
        for (j = 0; j < UINT8_BIT; j++) {
            if (r & 1) {
                r = (r >> 1) ^ CRCPOLY;
            } else {
                r >>= 1;
            }
        }
        mCrcTable[i] = (UINT16)r;
    }
}

STATIC
VOID
PutBits (
    IN INT32 n,
    IN UINT32 x
)
/*++
```

Routine Description:

Outputs rightmost n bits of x

Arguments:

n - the rightmost n bits of the data is used
 x - the data

Returns: (VOID)

```
--*/
{
    UINT8 Temp;

    if (n < mBitCount) {
        mSubBitBuf |= x << (mBitCount - n);
```

(continues on next page)

(continued from previous page)

```

} else {

    Temp = (UINT8)(mSubBitBuf \ (x >> (n -= mBitCount)));
    if (mDst < mDstUpperLimit) {
        *mDst++ = Temp;
    }
    mCompSize++;

    if (n < UINT8_BIT) {
        mSubBitBuf = x << (mBitCount = UINT8_BIT - n);
    } else {

        Temp = (UINT8)(x >> (n - UINT8_BIT));
        if (mDst < mDstUpperLimit) {
            *mDst++ = Temp;
        }
        mCompSize++;

        mSubBitBuf = x << (mBitCount = 2 * UINT8_BIT - n);
    }
}
}

```

```

STATIC
INT32
FreadCrc (
    OUT UINT8 *p,
    IN INT32 n
)
/*++

```

Routine Description:

Read in source data

Arguments:

p - the buffer to hold the data
n - number of bytes to read

Returns:

number of bytes actually read

```

--*/
{
    INT32 i;

    for (i = 0; mSrc < mSrcUpperLimit && i < n; i++) {
        *p++ = *mSrc++;
    }
    n = i;
}

```

(continues on next page)

(continued from previous page)

```
p -= n;
mOrigSize += n;
while (--i >= 0) {
    UPDATE_CRC(*p++);
}
return n;
}
```

```
STATIC
VOID
InitPutBits ()
{
    mBitCount = UINT8_BIT;
    mSubBitBuf = 0;
}
```

```
STATIC
VOID
CountLen (
    IN INT32 i
)
/*++
```

Routine Description:

Count the number of each code length for a Huffman tree.

Arguments:

i - the top node

Returns: (VOID)

```
--*/
{
    STATIC INT32 Depth = 0;

    if (i < mN) {
        mLenCnt[(Depth < 16) ? Depth : 16]++;
    } else {
        Depth++;
        CountLen(mLeft [i]);
        CountLen(mRight[i]);
        Depth--;
    }
}
```

```
STATIC
VOID
MakeLen (
    IN INT32 Root
```

(continues on next page)

(continued from previous page)

```
)
/*++
```

Routine Description:

Create code length array for a Huffman tree

Arguments:

Root - the root of the tree

```
--*/
{
    INT32 i, k;
    UINT32 Cum;

    for (i = 0; i <= 16; i++) {
        mLenCnt[i] = 0;
    }
    CountLen(Root);

    //
    // Adjust the length count array so that
    // no code will be generated longer than the designated length
    //
    Cum = 0;
    for (i = 16; i > 0; i--) {
        Cum += mLenCnt[i] << (16 - i);
    }
    while (Cum != (1U << 16)) {
        mLenCnt[16]--;
        for (i = 15; i > 0; i--) {
            if (mLenCnt[i] != 0) {
                mLenCnt[i]--;
                mLenCnt[i+1] += 2;
                break;
            }
        }
        Cum--;
    }
    for (i = 16; i > 0; i--) {
        k = mLenCnt[i];
        while (--k >= 0) {
            mLen[*mSortPtr++] = (UINT8)i;
        }
    }
}

STATIC
VOID
DownHeap (
    IN INT32 i
```

(continues on next page)

(continued from previous page)

```

)
{
    INT32 j, k;

    //
    // priority queue: send i-th entry down heap
    //

    k = mHeap[i];
    while ((j = 2 * i) <= mHeapSize) {
        if (j < mHeapSize && mFreq[mHeap[j]] > mFreq[mHeap[j + 1]]) {
            j++;
        }
        if (mFreq[k] <= mFreq[mHeap[j]]) {
            break;
        }
        mHeap[i] = mHeap[j];
        i = j;
    }
    mHeap[i] = (INT16)k;
}

```

```

STATIC
VOID
MakeCode (
    IN INT32 n,
    IN UINT8 Len[],
    OUT UINT16 Code[]
)
/*++

```

Routine Description:

Assign code to each symbol based on the code length array

Arguments:

n - number of symbols
 Len - the code length array
 Code - stores codes for each symbol

Returns: (VOID)

```

--*/
{
    INT32 i;
    UINT16 Start[18];

    Start[1] = 0;
    for (i = 1; i <= 16; i++) {
        Start[i + 1] = (UINT16)((Start[i] + mLenCnt[i]) << 1);
    }
}

```

(continues on next page)

(continued from previous page)

```

for (i = 0; i < n; i++) {
    Code[i] = Start[Len[i]]++;
}
}

```

STATIC

INT32

MakeTree (

IN INT32 NParam,

IN UINT16 FreqParm[],

OUT UINT8 LenParm[],

OUT UINT16 CodeParm[]

)

/*++

Routine Description:

Generates Huffman codes given a frequency distribution of symbols

Arguments:

NParam - number of symbols

FreqParm - frequency of each symbol

LenParm - code length for each symbol

CodeParm - code for each symbol

Returns:

Root of the Huffman tree.

--*/

```

{
    INT32 i, j, k, Avail;

    //
    // make tree, calculate len[], return root
    //

    mN = NParam;
    mFreq = FreqParm;
    mLen = LenParm;
    Avail = mN;
    mHeapSize = 0;
    mHeap[1] = 0;
    for (i = 0; i < mN; i++) {
        mLen[i] = 0;
        if (mFreq[i]) {
            mHeap[++mHeapSize] = (INT16)i;
        }
    }
    if (mHeapSize < 2) {
        CodeParm[mHeap[1]] = 0;
    }
}

```

(continues on next page)

(continued from previous page)

```
    return mHeap[1];
}
for (i = mHeapSize / 2; i >= 1; i--) {

    //
    // make priority queue
    //
    DownHeap(i);
}
mSortPtr = CodeParm;
do {
    i = mHeap[1];
    if (i < mN) {
        *mSortPtr++ = (UINT16)i;
    }
    mHeap[1] = mHeap[mHeapSize--];
    DownHeap(1);
    j = mHeap[1];
    if (j < mN) {
        *mSortPtr++ = (UINT16)j;
    }
    k = Avail++;
    mFreq[k] = (UINT16)(mFreq[i] + mFreq[j]);
    mHeap[1] = (INT16)k;
    DownHeap(1);
    mLeft[k] = (UINT16)i;
    mRight[k] = (UINT16)j;
} while (mHeapSize > 1);

mSortPtr = CodeParm;
MakeLen(k);
MakeCode(NParm, LenParm, CodeParm);

//
// return root
//
return k;
}
```